

Daily Scholar Papers Report — 2026-09-06

2026-09-06

Daily Scholar Papers Report — 2026-09-06

[Download PDF](#)

Window covered: 2026-09-05 → 2026-09-06 (Google Scholar alerts + user-curated self-emails, last 24 h) — which was empty — extended by the standing 7-day liveness sweep, which recovered a two-thread alert batch sent late on 4 September that no previous run had seen.

Executive Summary

A small window, and an unusually coherent one. Three papers arrived, and **not one of them is a detector**.

That is the thread. Each is a piece of scaffolding that makes somebody *else's* judgement trustworthy, and each is deliberately built to stop short of issuing a verdict.

JupyterDraw draws a picture of a machine-learning notebook and hands it to a human. It does not flag the data leakage. There is a well-established alternative — static detectors that match a fixed catalogue of known anti-patterns — and the paper declines it, on the grounds that a tool which issues a verdict is a tool a developer can accept uncritically. So it redirects attention instead. In a within-subjects experiment with 26 developers, that redirection raised the share of reviews catching **every** planted flaw from **29 % to 67 %**, at **+0.50 flaws per task** ($d_z = 0.69$, 95 % CI [0.21, 0.79], Wilcoxon $p = 0.005$) — and, importantly, without costing time or subjective workload.

Magma does the same thing one level up. A ground-truth fuzzing benchmark finds no bugs; it makes the claim “fuzzer A beat fuzzer B” mean something, by fixing in advance what the right answer was. Its authors have now written the retrospective on five years of that, which is the kind of document a field only gets when someone bothers.

And **combinatorial coverage on a RISC-V design** takes a criterion from software testing — t -way interaction coverage — and points it at hardware. A coverage criterion does not find a vulnerability either. It tells you which parts of the input space you have not yet looked at.

There is a second, sharper convergence, and it is about **abstention**. The strongest number in this window is not the detection gain. It is that when the generated diagrams were probed against code-derived ground truth, they answered 80 % and 84 % of questions correctly **with zero incorrect answers** — every miss was an explicit refusal to answer, not a confident wrong one. For an artefact whose usefulness is entirely conditional on being trusted, an 80 % score that never lies is a different object from an 80 % score that lies one time in five. Ground-truth benchmarking rests on the same instinct: know the boundary of what you are entitled to claim, and say nothing past it.

Outstanding: 1 · **Keep:** 1 · **Borderline High-Priority:** 1

A note on depth. `web_fetch` returned HTTP 429 on five attempts spread over 22 minutes, with one brief working window in between. **One paper was read in full** (Gomes et al., 1 120 lines of extracted text — every number below is transcribed from it). **One is documented at verified-publisher depth** — abstract, complete author list and licence confirmed on the arXiv abstract page, full text not retrieved. **One is at search-metadata depth only**, behind a publisher paywall; it is marked Borderline High-Priority because it could not be retrieved, not because the work looks marginal, and it heads the queue for the next run. Every card states its own evidence base and none claims more than its source supports. No paper in this window carried a CC-class licence, so no PDFs are mirrored here and no figures are reproduced.

Highlighted Papers

Title	Authors	Venue	Link
“Bring the image back!” An Empirical Study of Visual Summaries in Code Review	L. F. Gomes, X. Zhou, V. Hellendoorn, J. Aldrich, R. Abreu, D. Lo	Preprint, 2026 (CMU · SMU · Google DeepMind · Meta/FEUP)	author PDF · replication package
The Impact of Magma: A Ground-Truth Fuzzing Benchmark	A. Hazimeh, A. Herrera, S. Subramanian, T. Aman, S. Vaccino, Q. Liu, M. Payer	arXiv preprint, cs.CR, 28 Aug 2026	arXiv:2608.28016 · PDF
Combinatorial Methods for Chip Vulnerability Detection	R. Kuhn, C. Nokes, K. Amberiadis, M. Zuzak	IEEE Security & Privacy, 2026	IEEE 11672200

Papers

1.1 · CODE REVIEW · A picture of the pipeline doubles the rate of fully-correct reviews, 29 % → 67 %, and wins on primacy rather than preference

“Bring the image back!” An Empirical Study of Visual Summaries in Code Review

Authors: Luis F. Gomes (Carnegie Mellon University), Xin Zhou (Singapore Management University), Vincent Hellendoorn (Google DeepMind & CMU), Jonathan Aldrich (CMU), Rui Abreu (Meta & FEUP), David Lo (SMU)

Venue: Preprint, 2026. CCS concepts: Documentation; Multi-agent planning; Visualization systems and tools; Empirical studies in visualization.

Links. [author PDF](#) · [replication package](#). **Licence: none stated** — author-hosted preprint, so no mirror and no figures are reproduced here. **Evidence base: full text retrieved and read. Every number below is transcribed from the paper.**

The gap being targeted

Agents now write code faster than anyone reads it, and the failure mode that matters is not the one that crashes. Agentic LLMs iterate until the code executes and passes model-generated tests, so runtime exceptions are rare and *semantic* errors survive. In data science this is acute: a notebook with leaked

test data or a fabricated evaluation runs cleanly, prints plausible metrics, and is methodologically worthless. Jupyter makes it worse — cell-by-cell structure fragments the pipeline across independent execution units and offers no structural overview, so a reviewer must mentally reconstruct the data flow from raw input to evaluation before they can judge anything at all. And reviewers do not read every line; they work at the pipeline level. That level is spatial, and text is a poor medium for it.

The move: draw it, and then refuse to judge it

JupyterDraw is an MCP server — it runs inside whatever assistant the developer already uses — that turns a notebook into a whiteboard-style Excalidraw diagram: one box per meaningful step, coloured by role (data / process / model / eval / output), grouped into lanes, connected left-to-right by labelled data-flow arrows. The host model (Claude Opus 4.8 in the study) does the content selection; the visual vocabulary is fixed by injected instructions, which is what keeps diagrams comparable across notebooks. After rendering, the widget screenshots its own output and returns it to the model, enabling a self-review pass that repairs overlaps, lane violations and occluded labels.

The design decision worth stealing is what the tool *does not* do. There is an obvious alternative — static detectors that flag a fixed catalogue of known leakage anti-patterns — and the paper takes it seriously and declines it, because developers over-trust AI verdicts during review. A diagram redirects attention and leaves the unanticipated flaws for the human to judge. Sketching the pipeline is a way of making the reviewer do the thinking, cheaply.

The experiment

26 developers (13 male, 13 female; 17 doctoral, 9 master's; drawn from CMU, MIT, Brown, UC Irvine, IST and Porto, and from Amazon, Apple, Google DeepMind, Cohere and others), 52 review sessions, IRB-approved, in-browser Codespaces, 15-minute limit per notebook.

Two notebooks adapted from Kaggle — IMDB sentiment classification and blower-energy regression — each carrying **exactly two planted methodological flaws**: inconsistent train/test preprocessing, a model evaluated on synthetic rather than real predictions, features that are random noise, and a model tested on training data rather than the held-out batch. All four execute cleanly and produce optimistic metrics.

Every participant reviews one notebook with a **visual** overview (V) and one with a **textual** one (T), under a 4-group Latin square. The fairness control is the important part: the textual overview is *derived from the same diagram*, so both encode identical entities, dataflow and steps at the same abstraction level, and the only thing removed is visual-spatial encoding. The notebook code is byte-identical across conditions.

The primary model, transcribed from §4.4:

```
n_flaws ~ condition_v + notebook_imdb + order2 + (1|participant)
```

What the numbers say

Detection. Under T, reviewers caught 1.17 of two flaws (88 % caught at least one; **29 % caught both**). Under V, 1.67 (100 % caught at least one; **67 % caught both**). Of 24 participants, 13 did better with the diagram, 9 tied, 2 did worse. The paired difference is **+0.50 flaws** ($d_z = 0.69$, rank-biserial $r = 0.75$, Wilcoxon $p = 0.005$, bootstrap 95 % CI [0.21, 0.79] entirely above zero).

Robustness. The mixed model returns Condition (V) = 0.500, 95 % CI [0.22, 0.78], $p < 0.001$, while both controls sit at nothing — notebook $\hat{\beta} = 0.000$ ($p = 1.000$), order $\hat{\beta} = 0.083$ ($p = 0.561$). Marginal $R^2 = 0.178$, conditional $R^2 = 0.333$. Two alternative specifications agree: ordinal proportional-odds OR = 5.51 [1.63, 18.59] ($p = 0.006$); GEE logistic on complete detection OR = 5.02 [1.50, 16.79] ($p = 0.009$). Refitting across four ML-familiarity thresholds ($n = 26, 24, 22, 21$) always yields a V coefficient between 0.39 and 0.50 at $p \leq 0.010$.

The nulls, which are load-bearing. Three of four timing effects are non-significant with CIs spanning zero ($p > 0.3$), and the NASA-TLX composite does not differ (V 5.18 vs T 5.55, $p = 0.31$). The one significant timing result runs in V's favour: among reviewers who caught both flaws, V reached the second one about **1 min 57 s sooner** ($d = -0.60$, $p = 0.001$). Sessions ran a median of 14 minutes either way. So the detection gain is not bought with time or effort — it is free.

Mechanism, and the distinction that matters. Participants rated the diagram more helpful (median 5 vs 4 on a 1–6 scale, $\Delta = +1.38$, $d_z = 0.64$, $p = 0.011$) and more accurate (median 5 vs 4, $d_z = 0.47$, $p = 0.037$). But the attribution data is where the mechanism actually shows. Asked, per caught flaw, whether the overview or the code drove the catch: the *rate of any contribution at all* is statistically indistinguishable — **79 % (V) vs 72 % (T)**, $p = 0.56$. Both overviews get consulted. What differs is rank. The diagram was rated **above the code** as the primary detection route **39 % of the time versus 8 %** (OR = 7.50, $p = 0.008$). Under T the code stays dominant; under V the diagram competes with it on even terms. The advantage is primacy, not popularity.

Fidelity, and the number the paper under-sells. An LLM probe generated 25 code-grounded questions per notebook across five abstraction levels; a second model, shown only the rendered diagram, answered them, scored by exact match. Result: **80 % (IMDB) and 84 % (Energy) correct, with zero incorrect answers**. Every miss was an explicit abstention where the diagram did not depict that level of detail — near-perfect at pipeline structure, methodology and high-level workflow, degrading only at implementation minutiae like exact hyperparameter values. For a tool whose value the paper itself makes conditional on trust, a diagram that abstains rather than misleads is a categorically different artefact from one that is simply 80 % right.

Perception. 18 of 24 (75 %) preferred the visual overview. Eight said they would trust it only insofar as they could verify it against the code, and the most-requested improvements — code-linking, interactivity, an explicit key, a box-to-cell mapping — all serve exactly that. The title comes from what participants said unprompted on losing the diagram: *“It’s annoying, bring the image back!”* (P19).

On the paper’s honesty

§3.1 concedes, before the Threats to Validity section, that flaws of this severity are *“unlikely to appear in production code given the capabilities of modern LLMs”* — volunteering the main external-validity objection rather than waiting to be asked. The mitigation is empirical rather than rhetorical: three smaller models (Claude 3 Opus, GPT-4o-mini, GPT-4.1-nano) uniformly miss both flaws in both notebooks, and detection coding was replicated by a blind LLM coder over all 176 notes (Cohen’s $\kappa = 0.77$ at note level, 0.79 at participant level, > 86 % raw agreement).

Why it is worth your time

Three reasons, in ascending order of durability.

The immediate one: this is a clean, well-powered within-subjects experiment with a genuinely fair control condition — the textual overview derived from the same diagram, isolating visual-spatial encoding as the single manipulated variable — and it reports its nulls. If you write empirical SE papers, the control design is worth copying on its own.

The second: the delivery mechanism is MCP, not an IDE plugin. That makes the result immediately actionable rather than aspirational; the same pattern applies to any structural summary you would want an agent to hand a reviewer.

The third, and the one that will outlive the tool: the paper identifies *where* silent flaws live. Not inside function calls, where line-by-line reading would find them — in the **connections between steps**, which is precisely the structure a sequential representation flattens and a two-dimensional one preserves. That is a claim about representation, not about diagrams, and it generalises past notebooks.

As agents write more code and we read less of it, bringing pictures into the review loop offers a concrete way to keep developers accountable for what they ship.

1.2 · FUZZING · The Magma authors write their own five-year retrospective on what a ground-truth benchmark changed, and what they had to add

The Impact of Magma: A Ground-Truth Fuzzing Benchmark

Authors: Ahmad Hazimeh, Adrian Herrera, Srividya Subramanian, Thaqiya Aman, Sara Vaccino, Qiang Liu, Mathias Payer

Venue: arXiv preprint, cs.CR, submitted 28 August 2026. A short paper.

Links. [arXiv:2608.28016](https://arxiv.org/abs/2608.28016) · [PDF](#) · [DOI 10.48550/arXiv.2608.28016](https://doi.org/10.48550/arXiv.2608.28016). Original benchmark: [arXiv:2009.01120](https://arxiv.org/abs/2009.01120) · [SIGMETRICS 2021, doi:10.1145/3428334](https://doi.org/10.1145/3428334). **Licence:** arXiv non-exclusive distribution 1.0 — not CC, so no mirror and no figures here. **Evidence base:** verified publisher abstract, full author list, submission date and licence, read from the arXiv abstract page. Full text not retrieved (web_fetch rate limit). **Nothing below the abstract level is claimed.**

What it is

A short paper by the original Magma team, retrospective in kind rather than new in result. Per the abstract, it covers three things: **the motivation** for a ground-truth fuzzing benchmark, **the design**, and **the impact** since the SIGMETRICS 2021 release — plus a description of **extensions to the original benchmark**. The extensions are the part the abstract does not enumerate and the part a reader will actually want; that requires the full text.

Why a retrospective on this is worth the shelf space

The problem Magma was built for is the one every fuzzing paper still has to navigate. Compare two fuzzers by counting crashes and you are counting an artefact of deduplication heuristics. Compare them by counting CVEs found in the wild and you are measuring luck and target selection. Neither supports the sentence “A is better than B.” Ground truth — fixing, in advance and by construction, which bugs exist and how each is recognised — is what converts a fuzzing comparison from a demonstration into a measurement.

That places this paper alongside 1.1 and 1.3 in this window, and the parallel is exact: **a benchmark finds no bugs**. Its entire function is to make somebody else's claim checkable. This window happens to contain one such artefact at the level of a human reviewer, one at the level of a research community, and one at the level of a coverage criterion.

How to read it

As a bibliographic and historical document, not a technique paper — the depth ceiling is low enough that the missing full text costs less here than it would elsewhere. Read it for the extensions, for what five years of community use exposed about the original design, and as a model for how to write the retrospective on your own infrastructure. Benchmarks are the most under-credited artefacts in systems security precisely because the people who build them rarely write down what happened next.

Worth pairing with the ongoing work on re-validating revived ground-truth vulnerabilities — [arXiv:2503.19909](https://arxiv.org/abs/2503.19909) — which examines how well forward-ported bugs keep behaving like the originals, the standing question for any benchmark of this shape.

1.3 · HARDWARE TESTING · *t*-way combinatorial coverage, an SE testing construct, pointed at a RISC-V design

Combinatorial Methods for Chip Vulnerability Detection

Authors: R. Kuhn, C. Nokes, K. Amberiadis, M. Zuzak

Venue: IEEE Security & Privacy, 2026 (magazine article).

Links. [IEEE Xplore 11672200](https://ieeexplore.ieee.org/abstract/document/11672200) · [Scholar lookup](#). **Licence:** IEEE, all rights reserved — no mirror, no figures. **Evidence base:** Scholar-alert snippet plus search-metadata corroboration of venue and author group. Publisher page is paywalled; the abstract could not be verified at source and the full text was not retrieved. Flagged Borderline High-Priority on retrieval grounds, not on merit.

The stated claim

Per the alert snippet: adapting advanced software testing methods to digital hardware's inherent parallelism may improve fault and vulnerability detection, demonstrated on a RISC-V design with results described as high-quality and consistent. Nothing further is verifiable at this depth, and nothing further is asserted below.

Why it is flagged despite thin retrieval

The interesting direction here is the one of transfer. *t*-way combinatorial interaction coverage is a software-testing construct with a long and well-documented empirical history — most faults are triggered by interactions among a small number of parameters, so covering all *t*-way combinations for small *t* buys most of the fault-detection power of exhaustive testing at a fraction of the cost. Carrying that to RTL is not obviously sound: hardware's concurrency means the “parameters” whose interactions you are covering are not independent in the way a software API's arguments usually are, and what counts as a covered combination has to be redefined against a design that evaluates everything at once. Whether the empirical regularity survives that redefinition is a real question, and it is the question a reader should take to the full text.

The authors are the NIST combinatorial-testing group and collaborators, whose adjacent 2026 work applies mandatory-access-control models to hardware resource isolation in SoC verification — the same programme of importing software-assurance abstractions into hardware verification.

Status

Queued for re-read at full depth. NIST ACTS-group work is customarily posted to csrc.nist.gov, so an accessible version is likely to surface; if it does, this paper should be promoted out of Borderline and read properly.

Cross-Paper Synthesis

Three artefacts, none of which is a detector

The window is small enough that the pattern could be coincidence, and clear enough that it is worth naming anyway. Each of these three papers builds something that deliberately stops short of telling you the answer.

JupyterDraw could have flagged the data leakage. The technology exists — the paper cites static detectors that match a catalogue of known anti-patterns — and it explicitly declines, on the argument that a developer who over-trusts AI-generated *code* will also over-trust an AI-generated *verdict about* that code. The diagram redirects attention instead. Magma finds no bugs; it fixes what the right answer was, so that a comparison between two fuzzers becomes a measurement rather than an anecdote. A coverage criterion detects nothing at all; it partitions an input space and tells you which parts you have not visited.

What all three are is **infrastructure for someone else's judgement**. That is a category the field systematically under-credits relative to detectors and agents, and it is worth noticing when three instances of it turn up in one 32-hour window from three unrelated communities.

The abstention, which is the sharper convergence

There is a stronger version of the same observation, and it turns on a single clause in 1.1's evaluation.

The generated diagrams answered 80 % and 84 % of code-grounded questions correctly. The eye goes to the missing 20 %. The load-bearing clause is the next one: **zero incorrect answers**. Every miss was an explicit abstention — the diagram did not depict that level of detail, so no answer was given. Fidelity was near-perfect at pipeline structure, methodology and high-level workflow, and degraded only at implementation minutiae a structural diagram was never meant to convey.

That distinction is the whole ballgame for an artefact whose usefulness is conditional on being trusted, and 1.1's own qualitative data proves it: eight participants said the overview is useful only as long as it can be trusted, and the most-requested features were all mechanisms for *cheaply verifying* fidelity. A representation that is 80 % complete and never wrong can be audited by spot-check. A representation that is 80 % right and 20 % wrong cannot be audited at all, because you would have to check everything, at which point you may as well have read the code.

Ground-truth benchmarking is the same instinct wearing different clothes. Magma's premise is that you may only make comparative claims about the bugs you planted and can recognise; everything else is

outside what the instrument is entitled to say. Combinatorial coverage, likewise, makes a bounded claim — every t -way interaction has been exercised — and no claim whatsoever about what lies beyond t .

Knowing the boundary of what you are entitled to assert, and stopping there, is what makes all three of these artefacts usable. That is a design property, not a modesty. It is worth carrying into anything you build that a human is meant to trust without re-deriving.

Where the theme goes next: fidelity is the bottleneck, and nobody has solved it

1.1 leaves its own hardest problem open and says so. The diagram's value is conditional on faithfulness to the code, faithfulness was established here by manual inspection of two deliberately simple 15-minute notebooks plus an LLM probe, and five participants independently raised scalability as the open question — will a diagram of a real codebase be legible, and how would you know it was accurate?

That question is not specific to diagrams. It is the same question a benchmark faces when its forward-reported bugs drift from the originals they were derived from, and the same one a coverage criterion faces when the model of the input space stops matching the system. **Every artefact in this window buys trust by fixing a correspondence between a representation and a system, and none of them has a cheap way to check that the correspondence still holds.** The most-requested features in 1.1 — code-linking, box-to-cell mapping, an explicit key — are all attempts to make that check cheap. Whoever solves continuous, low-cost fidelity checking for generated representations solves it for all three layers at once.

A methodological note worth stealing

1.1's control condition deserves separate mention, because it is the kind of design decision that determines whether a result means anything. The obvious way to run “diagram vs text” is to generate a textual summary independently — and it would have been uninterpretable, because any difference could be attributed to one summary containing more information than the other.

Instead the textual overview is *derived from the diagram*. Same entities, same dataflow, same steps, same abstraction level; the only thing subtracted is visual-spatial encoding — colour, arrows, boxes, layout. The paper then confirms via LLM probing that the planted flaws are derivable from either representation alone, so both conditions are in principle sufficient. That reduces the experiment to a single manipulated variable, and it is why the +0.50 can be attributed to spatial encoding rather than to content.

If you are designing a comparison between two representations of the same thing, derive one from the other. It costs nothing and it is the difference between an effect and a confound.

Writing & Rationale Insights

Report your nulls, especially when they are what make your result mean something. 1.1 has one significant timing effect and three non-significant ones, and it leads with the nulls. This is not scrupulousness for its own sake — the nulls are what convert “reviewers caught more flaws” into “reviewers caught more flaws *for free*.” Without the flat NASA-TLX and the three zero-spanning timing CIs, the obvious rebuttal is that the diagram simply bought accuracy with time. A paper that buried them would have a weaker claim, not a stronger one.

Concede the main objection early, in your own words, with an empirical mitigation attached. 1.1 admits in the motivating-example section — well before Threats to Validity — that flaws this severe are unlikely in production code. Doing it there, unprompted, is what makes it read as calibration rather than damage control, and the mitigation offered is a measurement (three smaller models miss both flaws in both notebooks) rather than an argument. Reviewers forgive known limitations; they do not forgive discovering them themselves on page 8.

Distinguish “used” from “used first,” and let the data pick. The easy summary of 1.1 is “reviewers preferred the diagram.” The paper’s own attribution analysis refuses it: both overviews were credited at indistinguishable rates (79 % vs 72 %, $p = 0.56$), and the entire effect sits in *rank* — rated above the code 39 % of the time versus 8 %. Reporting the null alongside the effect is what makes the mechanism claim precise instead of merely favourable, and the precision is what makes it transferable to representations other than diagrams.

A percentage without an error breakdown is half a number. 80 % correct with zero incorrect answers and 80 % correct with 20 % wrong answers are different artefacts with different deployment properties, and only one of them is shippable to a reviewer who has been told to trust it. When you report accuracy on anything a human will rely on, report the shape of the residual — abstention or error — because that is what the reader actually needs to make a decision.

Write the retrospective on your own infrastructure. 1.2 exists because its authors sat down five years after release and wrote what happened. Benchmarks, harnesses and datasets are the most under-credited artefacts in systems research, and the reason is partly that nobody documents the second half of their life — what the community did with them, what broke, what had to be extended. If you have built one, the retrospective is a paper, and only you can write it.

State your evidence base per claim, not per document. Every card above says how deeply its source was actually read, and 1.3 says plainly that its abstract could not be verified at source. That is not a disclaimer ritual — it tells a reader which sentences they can cite and which they must go and check. The same discipline is what separates a literature review that can be built on from one that has to be redone.